```
import gi
gi.require_version('Gst', '1.0')
from gi.repository import Gst, GLib
import os
import numpy as np
import cv2
import hailo
import threading
import time
from queue import Queue, Empty

from RPLCD.i2c import CharLCD
from adafruit_servokit import ServoKit
from gpiozero import Device, DigitalInputDevice, OutputDevice
from gpiozero.pins.lgpio import LGPIOFactory
import ollama

from hailo_apps.python.core.common.buffer_utils import (
    get_caps_from_pad,
    get_numpy_from_buffer,
)

from hailo_apps.python.core.gstreamer.gstreamer_app import app_callback_class
from hailo_apps.python.pipeline_apps.pose_estimation.pose_estimation_pipeline import
GStreamerPoseEstimationApp

# Use lgpio for Raspberry Pi 5 GPIO

Device.pin_factory = LGPIOFactory()




# LCD setup

lcd = CharLCD(

    i2c_expander='PCF8574',

    address=0x27,

    port=1,

    cols=16,

    rows=2,

    charmap='A00',
```

```
    auto_linebreaks=True
)



# PCA9685 servo setup
kit = ServoKit(channels=16, address=0x40)
servo = kit.servo[0]

SERVO_POOR = 20
SERVO_FAIR = 70
SERVO_GOOD = 120
SERVO_EXCELLENT = 170



# GPIO setup
BUTTON_PIN = 17
TRIG_PIN = 14
ECHO_PIN = 15

BUTTON_ACTIVE_HIGH = True

button = DigitalInputDevice(
```

```
    BUTTON_PIN,
    pull_up=None,
    active_state=BUTTON_ACTIVE_HIGH,
    bounce_time=0.05
)

trig = OutputDevice(TRIG_PIN, initial_value=False)
echo = DigitalInputDevice(ECHO_PIN, pull_up=None, active_state=True)



# Distance settings
MIN_DISTANCE_CM = 100
MAX_DISTANCE_CM = 220

# Analysis settings
ANALYSIS_SECONDS = 8

lcd.clear()
lcd.write_string("System Started")
lcd.cursor_pos = (1, 0)
lcd.write_string("Loading...")
```

```
servo.angle = SERVO_FAIR




def lcd_show(line1, line2=""):
    lcd.clear()
    lcd.write_string(line1[:16])
    lcd.cursor_pos = (1, 0)
    lcd.write_string(line2[:16])




def wait_for_button():
    while not button.is_active:
        time.sleep(0.05)


    time.sleep(0.3)




def measure_distance():
    trig.on()
    time.sleep(0.00001)
    trig.off()
```

```
    timeout = time.time()
    while not echo.is_active:
        start_time = time.time()
        if time.time() - timeout > 0.03:
            return None

    timeout = time.time()
    while echo.is_active:
        end_time = time.time()
        if time.time() - timeout > 0.03:
            return None

    elapsed_time = end_time - start_time
    distance_cm = (elapsed_time * 34300) / 2
    return distance_cm


def wait_for_good_distance():
    while True:
        distance = measure_distance()

        if distance is None:
            lcd_show("Distance error", "Check sensor")
```

```
            time.sleep(0.3)
            continue

        if MIN_DISTANCE_CM <= distance <= MAX_DISTANCE_CM:
            lcd_show("Distance OK", str(round(distance, 1)) + " cm")
            time.sleep(1)
            return

        if distance < MIN_DISTANCE_CM:
            lcd_show("Too close", str(round(distance, 1)) + " cm")
        else:
            lcd_show("Too far", str(round(distance, 1)) + " cm")

        time.sleep(0.3)


def move_servo_by_category(category):
    if category == "Excellent":
        servo.angle = SERVO_EXCELLENT
    elif category == "Good":
        servo.angle = SERVO_GOOD
    elif category == "Fair":
```

```
        servo.angle = SERVO_FAIR
    else:
        servo.angle = SERVO_POOR



def angle_to_180(angle):
    if angle is None:
        return None

    if angle > 180:
        return 360 - angle

    return angle



def choose_analysis_mode(user_input):
    text = user_input.lower()

    if "body" in text or "streamline" in text or "back" in text or "posture" in text:
        return "body"

    return "elbow"
```

```
def ask_ollama_with_timeout(prompt, timeout_seconds=60):
    result_queue = Queue()

    def run_ollama():
        try:
            response = ollama.chat(
                model='llama3.2:1b',
                messages=[
                    {'role': 'user', 'content': prompt}
                ],
                options={
                    'num_predict': 60,
                    'temperature': 0.2
                }
            )
            result_queue.put(response['message']['content'])
        except Exception as e:
            result_queue.put("Ollama error: " + str(e))

    thread = threading.Thread(target=run_ollama, daemon=True)
    thread.start()

    try:
```

```
        return result_queue.get(timeout=timeout_seconds)
    except Empty:
        return "Ollama took too long, but the pose analysis still worked."



def get_ollama_feedback(mode, category, left_avg, right_avg):
    prompt = (
        "You are a swim coach. Give simple feedback for a high school swimmer. "
        "Keep it short, around 2 sentences. "
        "Analysis mode: " + mode + ". "
        "Left average angle: " + str(round(left_avg, 1)) + ". "
        "Right average angle: " + str(round(right_avg, 1)) + ". "
        "Category: " + category + ". "
        "Do not use bullet points."
    )

    return ask_ollama_with_timeout(prompt, 60)



def get_category_from_angles(left_angles, right_angles, mode):
    if len(left_angles) == 0 or len(right_angles) == 0:
        return "Poor", None, None
```

```
left_avg = sum(left_angles) / len(left_angles)
right_avg = sum(right_angles) / len(right_angles)

difference = abs(left_avg - right_avg)

if mode == "body":
    left_error = abs(180 - left_avg)
    right_error = abs(180 - right_avg)
    average_error = (left_error + right_error) / 2
    total_error = average_error + difference * 0.5

    if total_error <= 10:
        category = "Excellent"
    elif total_error <= 25:
        category = "Good"
    elif total_error <= 45:
        category = "Fair"
    else:
        category = "Poor"

else:
    if difference <= 10:
```

```
            category = "Excellent"
        elif difference <= 25:
            category = "Good"
        elif difference <= 45:
            category = "Fair"
        else:
            category = "Poor"

    return category, left_avg, right_avg



# Inside this function is where you place the rest of your code as usual
def custom_processing_thread(pose_estimator):
    # This sleep gives enough time for the HAT to fire up and start detecting
    time.sleep(2)

    while True:
        lcd_show("Ask terminal", "Type mode")

        user_question = input("\nWhat would you like to check today? Input: ")

        mode = choose_analysis_mode(user_question)
```

```
print("Selected mode:", mode)



if mode == "body":

    lcd_show("Mode: Body", "Press button")

else:

    lcd_show("Mode: Elbow", "Press button")



wait_for_button()



lcd_show("Button pressed", "Move to spot")

time.sleep(1)



wait_for_good_distance()



left_angles = []

right_angles = []



start_time = time.time()



while time.time() - start_time < ANALYSIS_SECONDS:

    if mode == "body":

        angle_left = pose_estimator.calculate_body_part_angle(
```

```
            'left_shoulder',
            'left_hip',
            'left_knee'
        )

        angle_right = pose_estimator.calculate_body_part_angle(
            'right_shoulder',
            'right_hip',
            'right_knee'
        )

    else:
        angle_left = pose_estimator.calculate_body_part_angle(
            'left_shoulder',
            'left_elbow',
            'left_wrist'
        )

        angle_right = pose_estimator.calculate_body_part_angle(
            'right_shoulder',
            'right_elbow',
            'right_wrist'
        )
```

```
    angle_left = angle_to_180(angle_left)
    angle_right = angle_to_180(angle_right)

    if angle_left is not None and angle_right is not None:
        left_angles.append(angle_left)
        right_angles.append(angle_right)

        line1 = "L:" + str(round(angle_left, 1)) + " R:" + str(round(angle_right, 1))
        lcd_show(line1, "Analyzing...")
    else:
        lcd_show("No pose found", "Stand in frame")

    time.sleep(0.2)

category, left_avg, right_avg = get_category_from_angles(left_angles, right_angles, mode)
move_servo_by_category(category)

if left_avg is not None and right_avg is not None:
    lcd_show("L:" + str(round(left_avg, 1)) + " R:" + str(round(right_avg, 1)), "Analysis done")

    print("\nAnalysis complete")
```

```
        print("Mode:", mode)
        print("Left average:", round(left_avg, 1))
        print("Right average:", round(right_avg, 1))
        print("Category:", category)

        print("\nGenerating Ollama feedback...")
        lcd_show("Ollama", "Thinking...")

        feedback = get_ollama_feedback(mode, category, left_avg, right_avg)

        print("\nOllama feedback:")
        print(feedback)

        lcd_show("Done", category)

    else:
        lcd_show("Analysis failed", "Try again")
        print("Analysis failed: not enough valid pose data.")

    time.sleep(3)
```

```
# The rest of the code starts here and handles the operation of the hat and all other neccesary calculations
# The hat should update all of its detection data 30 times a second.
class PoseDataManager:
    def __init__(self):
        """
        Manages pose estimation data across threads
        Allows safe access to the latest detection data
        """
        self.latest_detection_lock = threading.Lock()
        self.latest_detection = None
        self.latest_width = None
        self.latest_height = None

    def update_detection(self, detection, width, height):
        """
        Update the latest detection data thread-safely

        :param detection: Hailo detection object
        :param width: Frame width
        :param height: Frame height
        """
        with self.latest_detection_lock:
            self.latest_detection = detection
```

```
            self.latest_width = width
            self.latest_height = height

    def get_latest_detection(self):
        """
        Retrieve the latest detection data thread-safely

        :return: Tuple of (detection, width, height) or (None, None, None)
        """
        with self.latest_detection_lock:
            return (
                self.latest_detection,
                self.latest_width,
                self.latest_height
            )

class PoseEstimator:
    def __init__(self, pose_data_manager):
        """
        Initialize PoseEstimator with a PoseDataManager

        :param pose_data_manager: Shared data management object
```

```
    """
    self.pose_data_manager = pose_data_manager
    self.keypoints = self._get_keypoints()


def _get_keypoints(self):
    """Get the COCO keypoints correspondence map."""
    return {
        'nose': 0,
        'left_eye': 1,
        'right_eye': 2,
        'left_ear': 3,
        'right_ear': 4,
        'left_shoulder': 5,
        'right_shoulder': 6,
        'left_elbow': 7,
        'right_elbow': 8,
        'left_wrist': 9,
        'right_wrist': 10,
        'left_hip': 11,
        'right_hip': 12,
        'left_knee': 13,
        'right_knee': 14,
        'left_ankle': 15,
```

```
        'right_ankle': 16,
    }

def get_body_part_coordinates(self, body_part, significant_figures=4):
    """
    Get normalized coordinates for a specific body part from latest detection

    :param body_part: Name of the body part
    :param significant_figures: Number of decimal places to round to
    :return: Tuple of normalized (x, y) coordinates or None
    """
    # Get latest detection
    detection, width, height = self.pose_data_manager.get_latest_detection()

    if detection is None or width is None or height is None:
        return None

    # If no landmarks, return None
    landmarks = detection.get_objects_typed(hailo.HAILO_LANDMARKS)
    if len(landmarks) == 0:
        return None

    # Get bbox and points
```

```
    bbox = detection.get_bbox()
    points = landmarks[0].get_points()

    # Get the specific keypoint
    keypoint_index = self.keypoints[body_part]
    point = points[keypoint_index]

    # Convert keypoint coordinates into full-frame normalized coordinates
    norm_x = point.x() * bbox.width() + bbox.xmin()
    norm_y = point.y() * bbox.height() + bbox.ymin()

    # Clamp the values between 0 and 1, then round to specified significant figures
    norm_x = round(max(0, min(1, norm_x)), significant_figures)
    norm_y = round(max(0, min(1, norm_y)), significant_figures)

    return (norm_x, norm_y)

def calculate_body_part_angle(self, point_a_name, point_b_name, point_c_name):
    """
    Calculate angle between three body parts directly by name.

    :param point_a_name: First body part name
```

```
:param point_b_name: Vertex body part name

:param point_c_name: Third body part name

:return: Angle in degrees or None if coordinates can't be retrieved

"""

# Get coordinates for each body part

point_a = self.get_body_part_coordinates(point_a_name)

point_b = self.get_body_part_coordinates(point_b_name)

point_c = self.get_body_part_coordinates(point_c_name)



# Check if any coordinates are None

if any(point is None for point in [point_a, point_b, point_c]):

    return None



# Convert to numpy arrays

a = np.array(point_a)

b = np.array(point_b)

c = np.array(point_c)



# Calculate vectors

ba = a - b

bc = c - b



# Calculate angle using arctan2 for full 360-degree range
```

```
        angle = np.degrees(np.arctan2(np.linalg.det([ba, bc]), np.dot(ba, bc)))



        # Ensure the angle is between 0 and 360 degrees

        if angle < 0:
            angle += 360

        return angle



class user_app_callback_class(app_callback_class):

    def __init__(self, pose_data_manager):

        """

        Initialize with a PoseDataManager



        :param pose_data_manager: Shared data management object

        """

        super().__init__()

        self.pose_data_manager = pose_data_manager



def app_callback(element, buffer, user_data):

    # In the newer Hailo Apps version, buffer is passed directly into the callback

    if buffer is None:

        return



    # Get the caps from the source pad
```

```
    pad = element.get_static_pad("src")
    format, width, height = get_caps_from_pad(pad)

    # Get the detections from the buffer
    roi = hailo.get_roi_from_buffer(buffer)
    detections = roi.get_objects_typed(hailo.HAILO_DETECTION)

    # Find the person detection
    person_detection = None
    for detection in detections:
        if detection.get_label() == "person":
            person_detection = detection
            break

    # If a person is detected, update the shared data
    if person_detection is not None:
        user_data.pose_data_manager.update_detection(person_detection, width, height)
    return

if __name__ == "__main__":
    # Create PoseDataManager first
    pose_data_manager = PoseDataManager()
    # Create an instance of the user app callback class with pose_data_manager
    user_data = user_app_callback_class(pose_data_manager)
```

```
# Create pose estimator
pose_estimator = PoseEstimator(pose_data_manager)

# Start the custom processing thread
processing_thread = threading.Thread(
    target=custom_processing_thread,
    args=(pose_estimator,),
    daemon=True

)

processing_thread.start()

# Run the GStreamer pipeline

app = GStreamerPoseEstimationApp(app_callback, user_data)
app.run()
```